Session 04 - Neural Networks

Convolutional Neural Networks (CNNs)

The MNIST Dataset will be used as an example later but a good interactive example of CNNs can be found here:Basic Convnet for MNIST.

> **Convolutional Neural Network - Wikipedia**
>
> "A convolutional neural network (CNN, or ConvNet) is a class of artificial neural network (ANN), most commonly applied to analyze visual imagery."
>
> - They are inspired by the organization of the animal visual cortex and are particularly effective at identifying **patterns in images.**

The Convolutional Process

At the core of a CNN is the **convolution operation**. This involves a `filter` (also known as a kernel) that slides over the input image. For each position, the filter performs a dot product with the underlying image pixels, producing a single value in the output feature map. This process helps in extracting features such as edges, textures, and patterns.

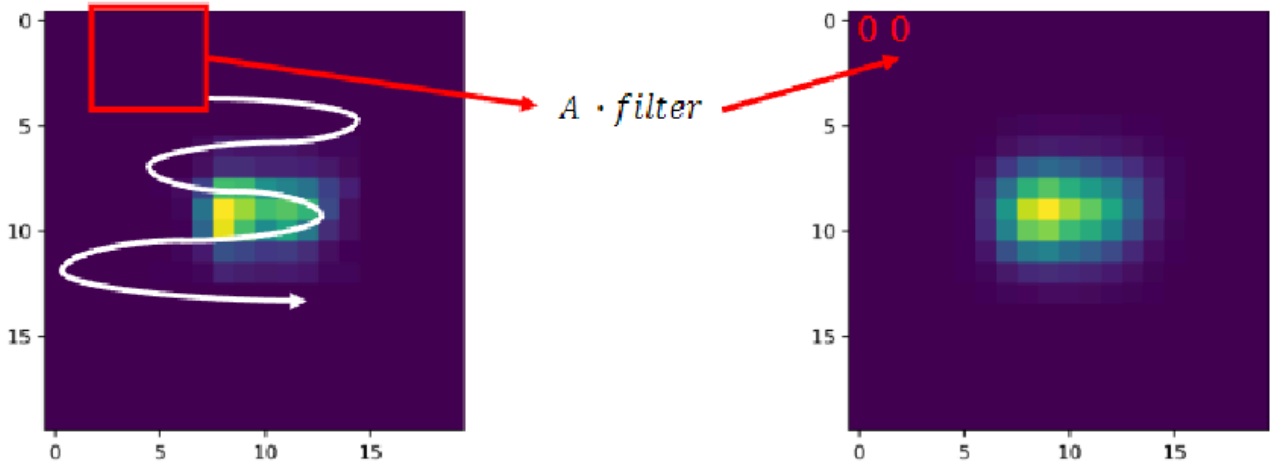
> **Convolution is Key!!**
>
> The **convolution operation** (in german: Faltung) refers to an operation that **blends two functions** together; so the a *signal* (like an image) and a *kernel* (a filter) → into a third function (the output).

> **⚠ Convolution is the foundation of CNNs.**
>
> Without convolution, a CNN would lose its ability to focus on **local features**, which is critical for object detection, facial recognition, and medical image analysis.

> Here is an awesome interactive Online Tool to visualize how the Kernal works. There different kernels / filters can be applied to detect different features!

The filter itself is a small matrix of weights that detects specific features.
See in the image below the `filter`:

$$filter = \begin{bmatrix} 0.5 & 0.5 & 0.5 \\ 0.5 & 1 & 0.5 \\ 0.5 & 0.5 & 0.5 \end{bmatrix}$$

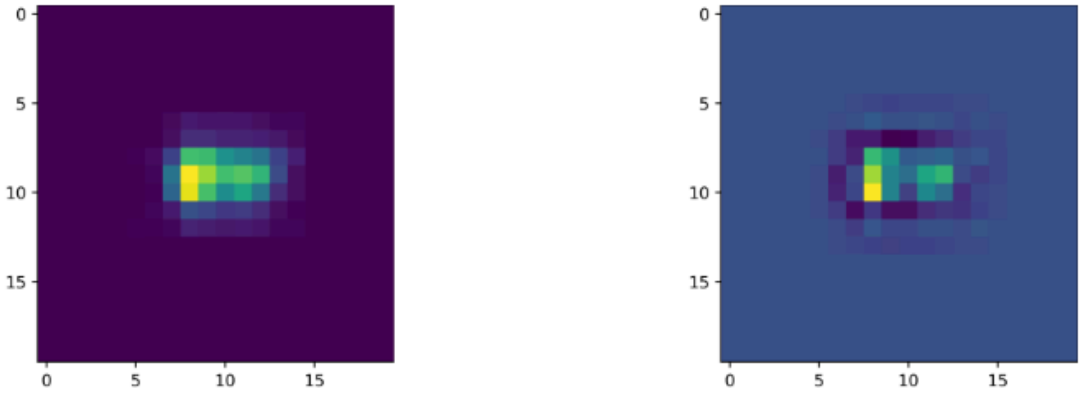


> This filter here enhances certain features or patterns within the image by giving more weight to the central pixel.

The transformed output on the right is called a feature map. This transformation highlights relevant features from the input.

Filter – Edge Detection Example

$$filter = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$



> Example for edge detection filter applied to the same input image

---

Convolutional Layer (with Code Examples)

The `Conv2D` layer in libraries like TensorFlow/Keras is fundamental to CNNs. It learns features by convolving filters over the input.

Based on the lecture and on Wikipedia

```
tf.keras.layers.Conv2D(filters, kernel_size, strides=(1, 1), padding='valid', …)
```

During the training process, the model learns the optimal weights (`w`) for each filter. A filter can be visualized as a small matrix of weights:

$$filter = \begin{pmatrix} w_{0,0} & \cdots & w_{0,m} \\ \vdots & \ddots & \vdots \\ w_{n,0} & \cdots & w_{n,m} \end{pmatrix}$$

Key parameters (*see code above*) of a convolutional layer include:

- **Filter Count (`filters`)**: Determines the number of feature maps produced by the layer. Each filter learns to detect a specific pattern (e.g., edges, textures, corners, stripes, ...).
- **Kernel Size (`kernel_size`)**: Defines the dimensions of the sliding window (filter). Common sizes include (3,3) or (5,5).
- **Step Length (`strides`)**: Specifies how many pixels the filter moves across the input image. A stride of (1,1) means the filter moves one pixel at a time, resulting in a larger output feature map. Larger strides reduce the output size.
- **Padding (`padding`)**: Controls how the borders of the input are handled.
  - `'valid'` : No padding, meaning the filter only moves across valid input regions. This reduces the output size. This means the border of an image is not 'scanned'
  - `'same'` : Pads the input so that the output feature map has the same dimensions as the input, assuming a stride of (1,1). Meaning: it pretends there are extra Pixels around the border so the output can keep the the same size as the input and no information is lost.

Constraints

Convolutional layers require and produce data with specific shapes (e.g., `(height, width, channels)` for images).

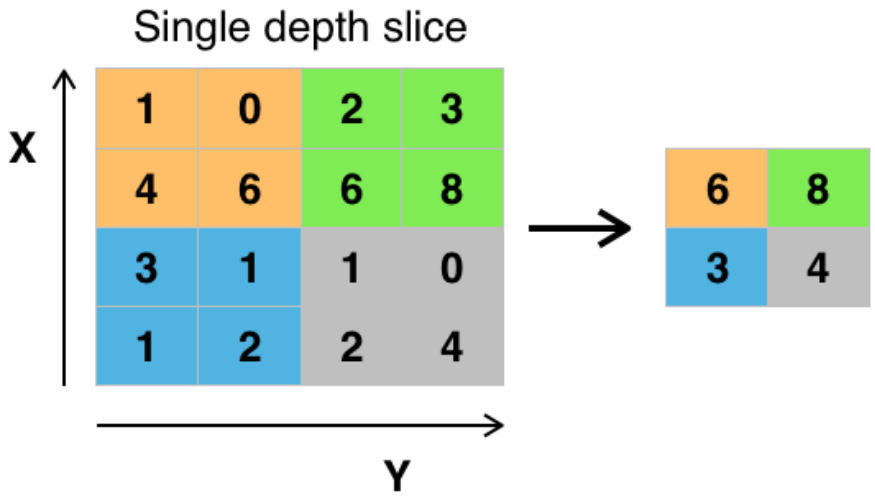
Pooling Layer

Pooling layers are used to reduce the spatial dimensions (height and width) of the feature maps, which helps in reducing the computational complexity and controlling overfitting.

Eli5

Pooling is like zooming out on a very detailed photo. We're not trying to keep every little pixel but just the most important parts. And **max-pooling** looks at a section of the given image and asks: "what is the most important pixel (e.g. highest value) here?" and keeps just that one (*see image below*)

> Max Pooling as a concept is rather simple, but here is a nice tool to play with the different parameters of max pooling, that also shows how pooling can add robustness to the learing process, when the filters like 'vertical stride' are set higher the input is shifted dramatically.

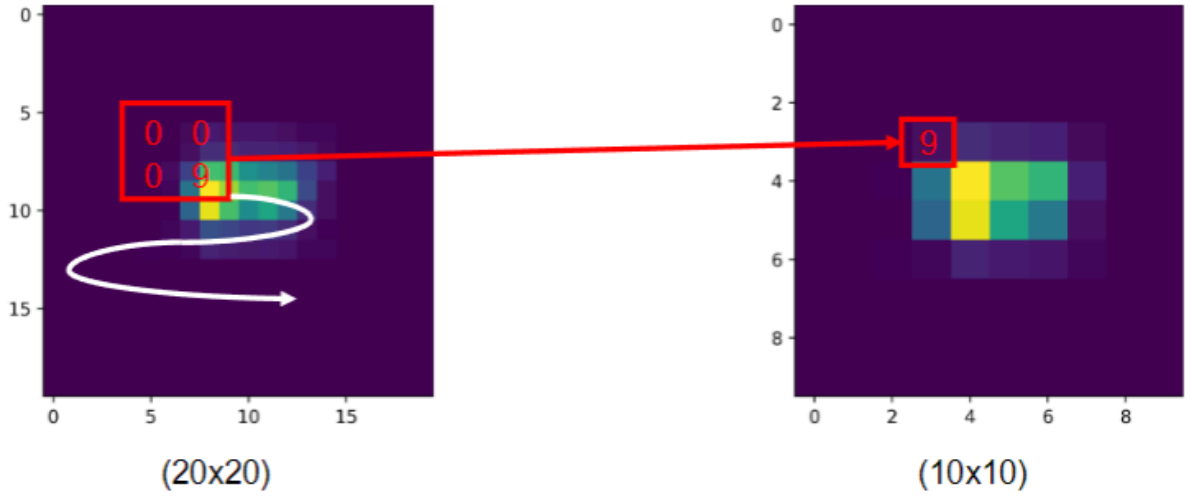


> Max pooling with a 2x2 filter and stride = 2

Max Pooling Layer

The Max Pooling layer is a common type of pooling.

- **Kernel Size (`kernel_size`)**: Similar to convolutional filters, a kernel (e.g., (2,2)) slides over the input.
- **Max Value within the Filter**: For each window defined by the kernel, the layer outputs only the maximum value, discarding the rest.
  This process downsamples the feature map, making the model more robust to small translations in the input.



Why do we do pooling, if we just reduce the given information?

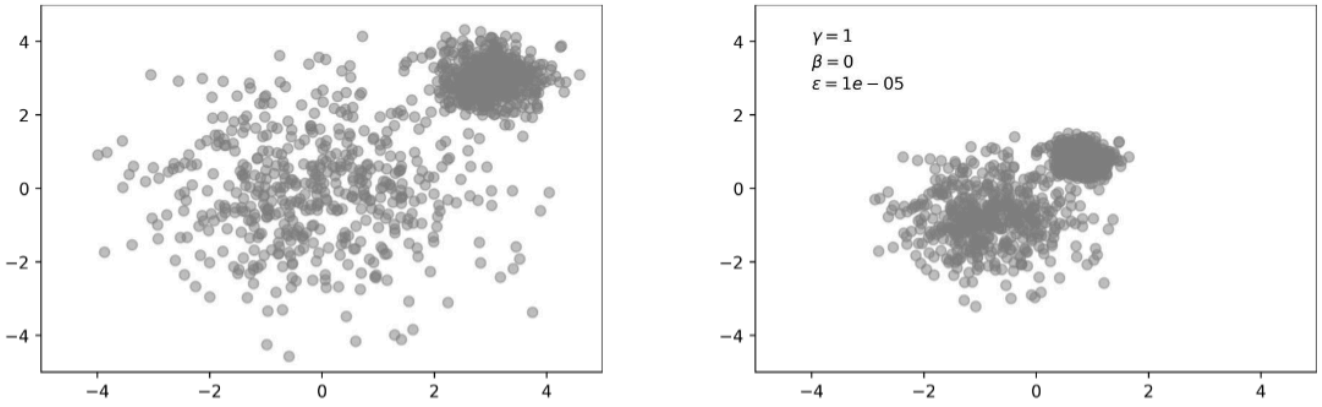
- It makes the data **smaller**, which means faster and cheaper computation.
- It helps the model **focus on the strongest signals**, like corners, edges, or patterns.
- It adds some **translation robustness**. So if a handwritten “5” is slightly shifted left or right, the network still recognizes it as a “5.”

Batch Normalization

Batch Normalization is a technique used to improve the training stability and performance of deep neural networks.

Good read: Batch Norm Explained Visually – How it works, and why neural networks need it

```
tf.keras.layers.BatchNormalization()
```



> In the example (left) with two features we can sense that they are on drastically different scales. Since the network output is a linear combination of each feature vector, this means that the network learns weights for each feature that are also on different scales. This could lead to the large feature might simply drowning out the small feature.

Usecase:

- **Improves Training**: It normalizes the inputs to each layer, effectively standardizing the mean and variance of the activations within a mini-batch.
- **Often Speeds up Training**: By stabilizing the input distribution for each layer, it allows for higher learning rates and faster convergence during training.

Effect of Batch Normalization

Batch Normalization addresses the problem of **Internal Covariate Shift**, where the distribution of activations in a network changes as the parameters of previous layers are updated.
If the training data points are spread out, forming two distinct clusters (*like in the image above*, forming two groups).
Batch Normalization works by rescaling and re-centering these activation distributions, effectively moving all these points closer together within their respective clusters.
This makes the optimization landscape smoother and easier for the model to learn.

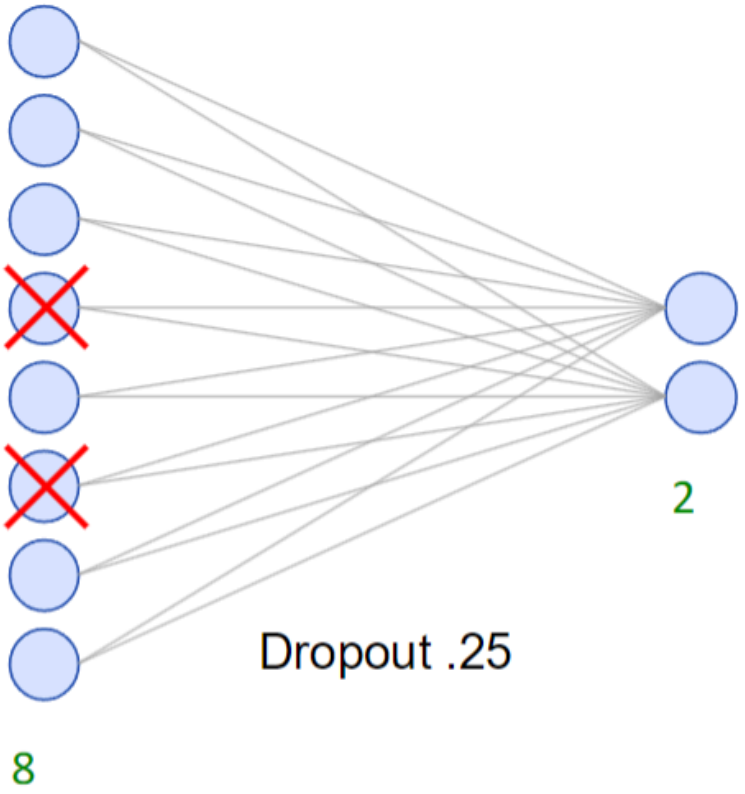
Dropout

Dropout is a regularization technique primarily used to **reduce overfitting** in neural networks.

Good read: Unveiling the Dropout Layer: An Essential Tool for Enhancing Neural Networks

With dropout, certain nodes in a NN are set to the value zero in a training run, i.e. removed from the network. Thus, they have no influence on the prediction and also in the backpropagation. Thus, a new, slightly modified network architecture is built in each run and the network learns to produce good predictions without certain inputs.

When using a dropout layer, a so-called dropout probability must also be specified. This determines how many of the nodes in the layer will be set equal to 0. In the example below we set the dropout probability or also called dropout-rate to 25%, so every 4^{th} node is set to 0!




```
tf.keras.layers.Dropout(rate=0.25)
```

- **Reduces Overfitting**: During training, a specified percentage (`rate` , e.g., 0.25 for 25%) of neurons in a layer are randomly "dropped out" (i.e., their outputs are temporarily ignored), along with their connections.
  - This forces the network to learn more robust features because it cannot rely on any single neuron or a small set of neurons always being active.
  - In this way, the danger of overfitting can be reduced in deep neural networks, since the neurons do not form a so-called adaptation among themselves, but recognize deeper structures in the data.
  - The dropout layer can be used in the input layer as well as in the hidden layers.
  - However, once the training has been trained out, the dropout layer is no longer used for predictions. However, in order for the model to continue to produce good results, the weights are scaled using the dropout rate.

Why Dropout Works

Considering a neural network with 8 input and 2 output nodes.
If, for instance, 2 input nodes are "crossed out" (*see image above*), the network is forced to find alternative paths and feature combinations to make predictions.
This prevents the model from becoming overly dependent on specific features or co-adaptations between neurons, thus improving its generalization capability to unseen data.

What Can We Do Now?

By combining these powerful components...

1. **Convolutional Layers**: for feature extraction - like shapes and edges

2. **Pooling Layers** for dimensionality reduction - to only focus on the hardest edges (and reduce image size, making everything faster)
3. **Batch Normalization** for stable training - to keep the learning process stable and fast by standardizing activations
4. and **Dropout** for regularization and preventing overfitting - to make sure all edges are equally important

... we can construct sophisticated deep learning models capable of solving complex tasks, particularly in computer vision.

Example Model Explanation

This `tf.keras.Sequential` model demonstrates a typical Convolutional Neural Network architecture, combining the layers discussed above for a classification task.

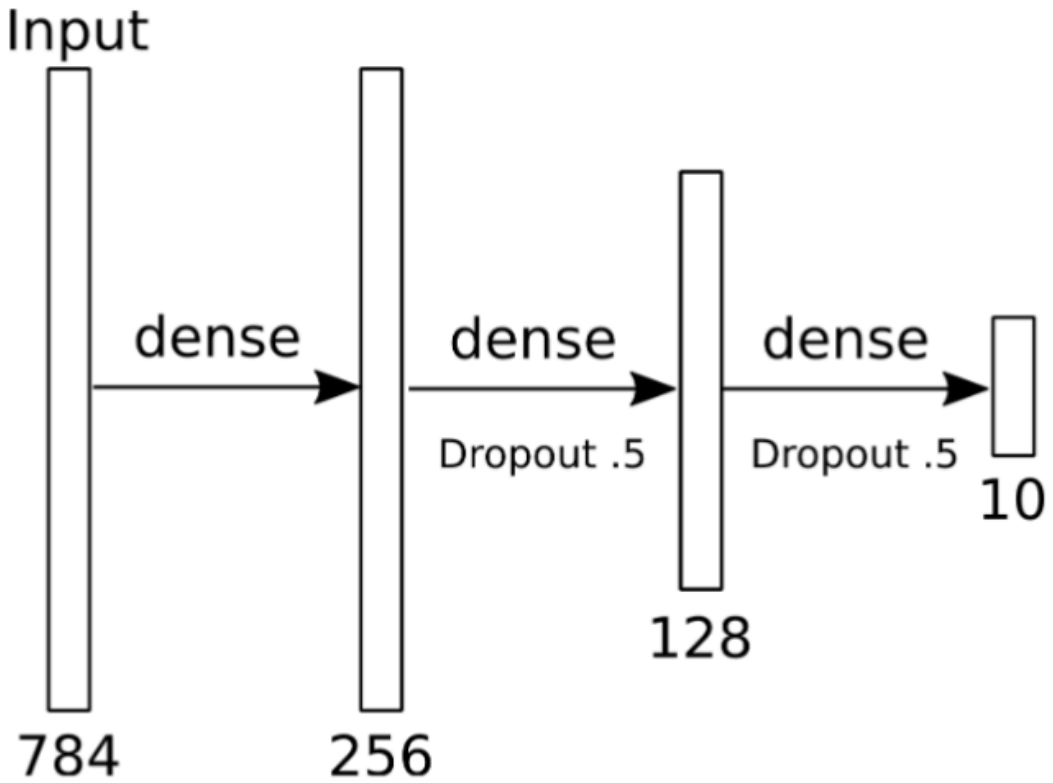
```
# Initialize a sequential model, a linear stack of layers
model = Sequential()

# Define the input shape: 28x28 pixels, 1 channel (grayscale)
model.add(InputLayer((28,28,1), name="InputLayer"))

# Add a 2D convolutional layer with 128 filters, 3x3 kernel, ReLU activation, padding to keep
dimensions
model.add(Conv2D(filters=128, kernel_size=(3,3), activation='relu', padding='same'))

# Add max pooling to downsample the feature maps (default 2x2 pool size)
model.add(MaxPool2D())

# Add dropout layer to randomly disable 25% of neurons during training to reduce overfitting
model.add(Dropout(rate=0.25))

# Add another 2D convolutional layer with 64 filters, 3x3 kernel, ReLU activation, same padding
model.add(Conv2D(64, kernel_size=(3,3), activation='relu', padding='same'))

# Add max pooling again to further downsample feature maps
model.add(MaxPool2D())

# Add batch normalization to stabilize and speed up training by normalizing activations
model.add(BatchNormalization())

# New layer Flatten needed after Conv2D to use a Dense next, because Conv2D returns a multi-
dimensional tensor. This flattens the multi-dimensional tensor output to a 1D vector for the
next dense layers
model.add(Flatten())

# Add a dense (fully connected) layer with 128 neurons and ReLU activation
model.add(Dense(128, name="HiddenLayer1", activation='relu'))

# Add a dense layer with 64 neurons and ReLU activation
model.add(Dense(64, name="HiddenLayer2", activation='relu'))

# Output dense layer with 2 neurons for binary classification using softmax activation
model.add(Dense(2, name="OutputLayer", activation='softmax'))
```

Optimizer and Hyperparameter

"**Impact of hyperparameters on a model**" means we're looking at how different **settings** (like the number of neurons, learning rate, dropout rate, etc.) affect how well a machine learning model learns and performs.

- If the **temperature** is too high → it halucinates.
- If the **hidden layers** are to broad and not reduced → it might loose focus.

> The **MNIST dataset** (Modified National Institute of Standards and Technology database)
>
> "A large database of **70,000 handwritten digits** (0 to 9), with 60,000 for training and 10,000 for testing. Each digit is represented as a **28×28 grayscale image**."
> *(LeCun et al., 1998)*



Analyzing the MNIST Dataset with a simple sodel:

- **Input (784)**:
  Each 28×28 pixel image is flattened into a long row of 784 numbers (28×28 = 784). These are the pixel brightness values.
- **Dense(256)**:
  First fully connected layer with 256 neurons. Every input is connected to every neuron.
  This layer starts learning patterns, maybe strokes or loops from digits. It is using the sobel-kernel.
- **Dense(128) with Dropout 0.5**:
  Another dense layer with 128 neurons, but **with dropout**, so only every second neuron is used. This layer refines the learned features further, like distinguishing a '5' from a '6'.
- **Dense(10) with Dropout 0.5**:
  Final output layer. There are **10 neurons**, one for each digit (0–9).
  The model outputs **10 probabilities**, whichever is highest is the model's guess!
  Again with a dropout of 0,5 before the output!

A little different in the execution but still nice interactive 'explanation': Basic Convnet for MNIST.*

> **What if we had a dropout of 0.5 after the last output layer?**
>
> We'd be randomly "turning off" 50% of the **10 output neurons**, which means **some digits might never be predicted** during training! That would be bad!

For a deeper understanding read: # Convolutional Neural Network (CNN): A Complete Guide

---

Optimizer

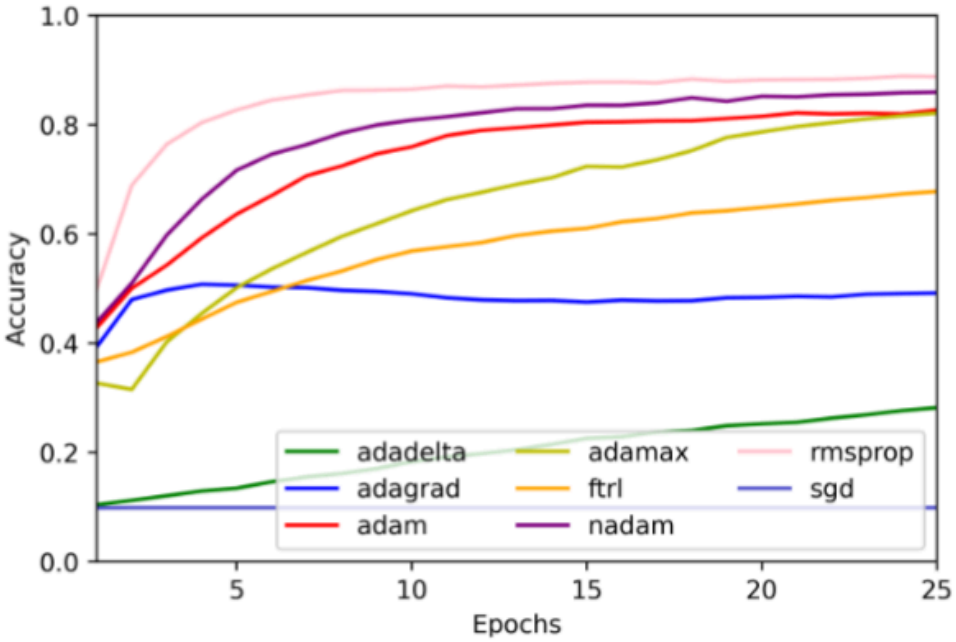
> What is an Optimizer?
>
> An **optimizer** is an algorithm that adjusts the weights and biases of a neural network during **backpropagation**, with the goal of minimizing the **loss function**; i.e., reducing the model's prediction error.

It evaluates how well the model performed (based on the loss), then **slightly changes the weights** to make the next prediction better.
It looks at the weights after each attempt of predicting a number. and changes the values slightly, to hopefully improve the prediction.

Common Optimizers

Optimizer	Description
`SGD` (Stochastic Gradient Descent)	Simple, slower, and sensitive to learning rate.
`Adam`	Adaptive Moment Estimation – fast, stable, and popular (e.g., for MNIST).
`Adagrad`	Adapts learning rates individually, good for sparse data.
`RMSprop`	Good for recurrent networks; adapts learning rate based on recent gradients.
`Nadam`	Adam + Nesterov momentum. Often performs well out-of-the-box.
`Adadelta`	Variation of Adagrad with fewer parameters.

Optimizer Test: Performance on MNIST Subset



> A line plot comparing different optimizers (Adam, Adagrad, SGD, RMSprop, etc.) based on their accuracy over training epochs.

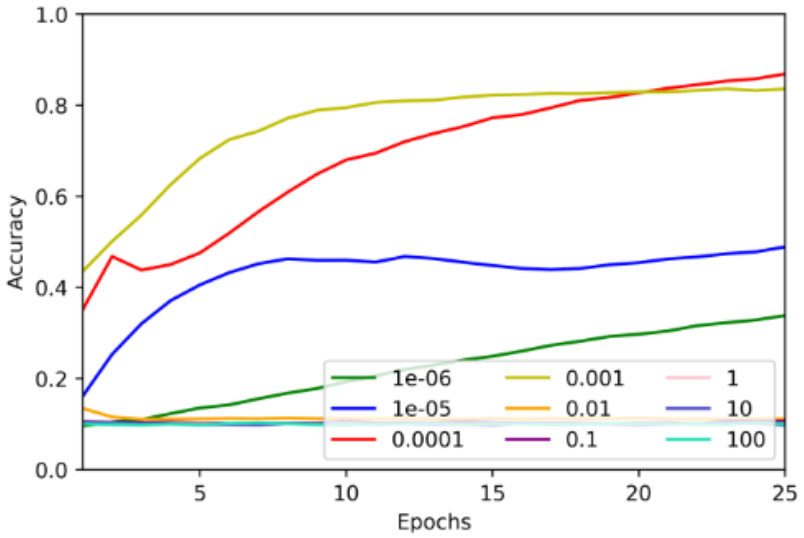
Interpretation:

- Most optimizers (like Adam, RMSprop, Nadam) show a **steep increase in accuracy** early in training and then plateau → **this is ideal.**
- Others (like Adagrad, Adadelta) **improve slowly** or even **decline**, indicating they might not perform as well with the chosen architecture and dataset.
- This shows that **choice of optimizer strongly affects learning speed and final accuracy**, even with identical model architectures.

Learning Rate

> Learning Rate
>
> The **learning rate** controls how much the optimizer changes the weights during each update. It's a critical hyperparameter that affects convergence speed and stability.

Learning Rate Effects (using Adam)



2 dense layers with dropput tested on a subset of the MNIST dataset

> Accuracy curves over epochs for different initial learning rates.

Conceptual Interpretation:

- **Too high a learning rate** can lead to **unstable training**, where the model overshoots the optimal values.
- **Too low a learning rate** results in **slow learning** and possibly getting stuck in local minima.
- A **moderate value like 0.0001 or 0.001** often hits the "sweet spot", combining stability and speed.

> **Learning Rate "Speed"**
>
> It's best not to become "too accurate too quickly", as that often leads to **overfitting**.

Adaptive Learning Rate Strategies

These methods help the optimizer **adjust the learning rate dynamically** during training:

- **Decay over time:** Gradually reduces the learning rate as training progresses.
- **Decay on plateau:** Lowers the learning rate when performance stops improving.
- **Momentum:** Remembers previous gradient directions to **accelerate training** and avoid getting stuck.

Loss Functions

Loss functions provide a measure of how **well a model's predictions match the expected output**. During training, optimizers (*see above*) use these values to adjust model parameters.

Classification vs. Regression Loss Functions

Loss functions vary depending on the **type of output** a model is predicting:

Problem Type	Common Loss Functions
Classification	Binary Cross Entropy, Categorical Cross Entropy, Hinge Loss
Regression	Mean Absolute Error (L1), Mean Squared Error (L2), Mean Squared Logarithmic Error

> **Why the right loss matters**
>
> "Choosing an appropriate loss function is essential: it directly affects how the model learns and what it prioritizes."

Binary Cross Entropy (BCE)

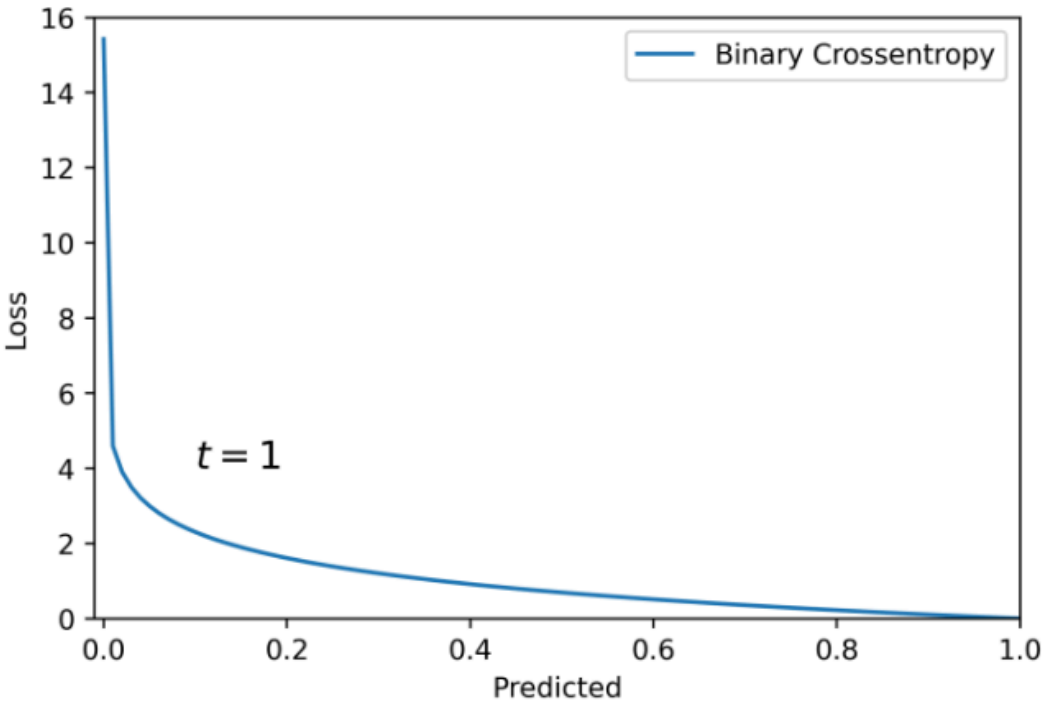
Binary Cross Entropy is used when the model predicts a **probability for two classes** (binary classification).

> 🔍 **Binary Cross Entropy (BCE):**
> BCE compares the predicted probability (between 0 and 1) to the true label (0 or 1). The more incorrect the prediction, the higher the loss.

- BCE penalizes confident but wrong predictions more harshly.

> **BCE Intuition:**
>
> If the model predicts 0.01 for a class that is actually 1, the penalty is much stronger than if it had predicted 0.4.

Interpreting the BCE Graph for Classification



The graph shows:

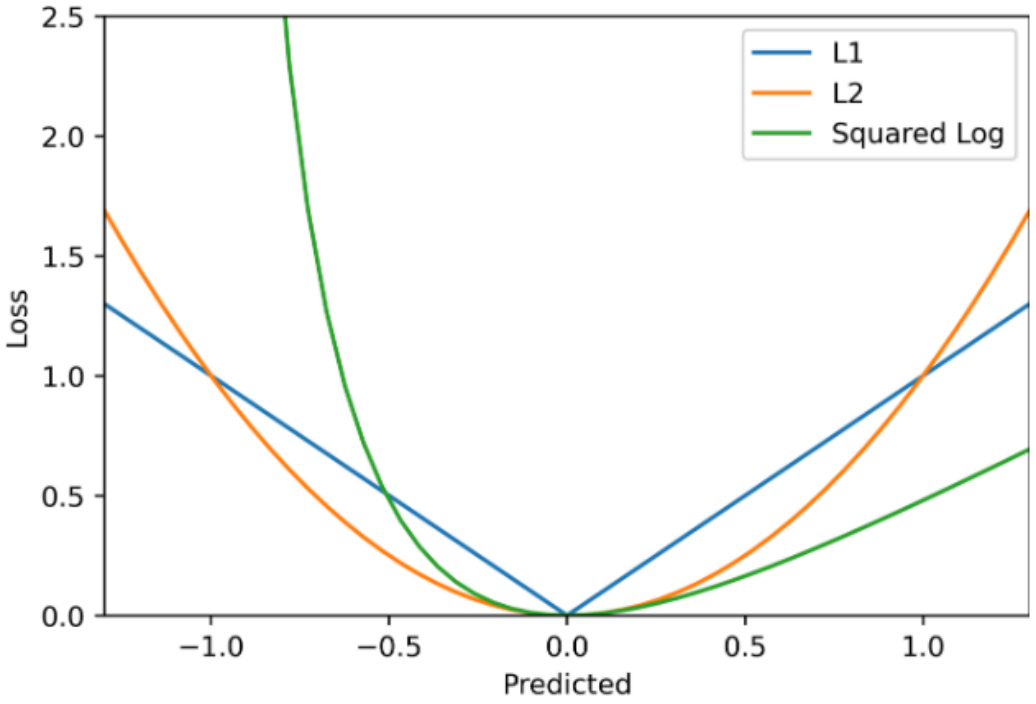
- **X-axis:** Predicted value (0 to 1)
- **Y-axis:** Loss value
- Loss is **very high** for predictions near 0 when the actual label is 1
- The curve **drops steeply** from left to right and **flattens out** near 1

This means that **correct predictions** (e.g. predicting 0.9 when label is 1) result in low loss, while **confident incorrect predictions** (e.g. predicting 0.01 when label is 1) result in high loss.

> ***Why is BCE not symmetric around 0.5?***
>
> Because it is calculated for a true label of 0 or 1. So, the steep drop corresponds to one class. If the label was 0, the curve would be mirrored.
> So the BCE curve is not symmetric around 0.5, because it's always shown for a fixed true label (usually $y = 1$). Each class has its **own loss curve**, and BCE reflects that in its directionality.

Loss Functions for Regression

When predicting continuous outputs, different loss functions are used:



🔵 Mean Absolute Error (MAE / L1 Loss)

- Measures the average absolute difference between prediction and target.
- Less sensitive to outliers.
- Gradient is constant, which can make learning slow.

🟠 Mean Squared Error (MSE / L2 Loss)

- Measures the average squared difference between prediction and target.
- Heavily penalizes larger errors.
- More sensitive to outliers.

🟢 Mean Squared Logarithmic Error (MSLE)

- Like MSE, but operates on the logarithm of predictions.
- Useful when **relative** differences matter more than **absolute** ones.

> 🔍 **MSLE Use Case:**
> MSLE is often used in **growth predictions** like population or income where large values must be scaled.

When to use L1 vs L2:

- Use L1 when outliers exist and should not dominate training
- Use L2 when we want to penalize large deviations heavily

Specialized Losses for Pixel-Level Predictions

For tasks like **image generation**, standard loss functions often result in **blurry outputs**. More advanced losses combine different aspects:

> 🔍 **Pixel-wise Loss:**
> Traditional loss function measuring exact pixel differences (like MSE)

> 🔍 **Feature Matching (FM):**
> Instead of comparing raw pixels, this method compares **features** extracted from intermediate layers of a network (e.g., VGG).

This helps preserve **texture and perceptual similarity**, rather than pixel-perfect accuracy.

Averaging is smoothing

it is like like averaging several slightly different photos: the result becomes soft and lacks sharp detail.

Why do pixel-wise losses lead to blurry outputs?

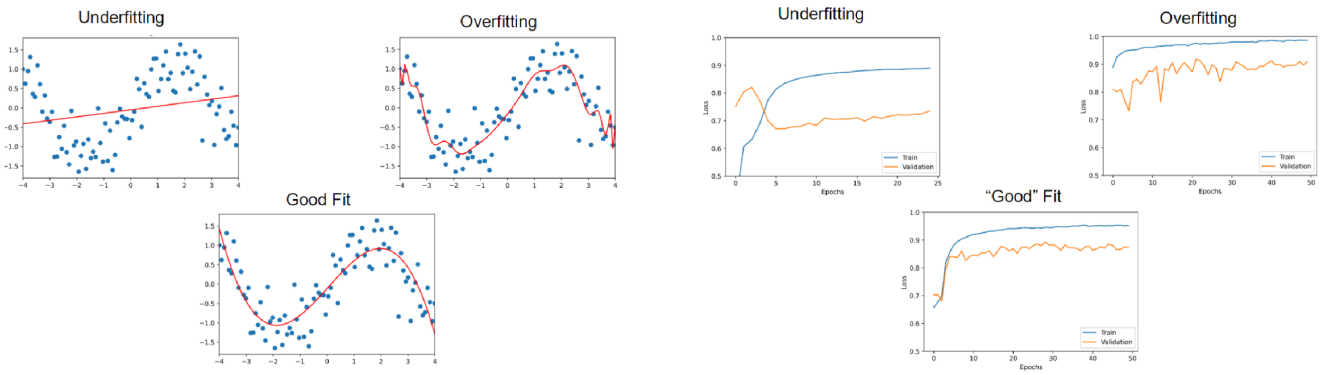
Pixel-wise losses (like MSE or MAE) measure the difference between each individual pixel of the prediction and the ground truth. When there are **multiple plausible correct outputs** (e.g. several valid ways to generate a face or a handwritten number), the model tends to **average** them to minimize the error.

This averaging smooths out differences and removes high-frequency details like edges or textures resulting in a **blurry image**.

Feature Matching (FM) loss helps retain those details by comparing deeper **features** (e.g. extracted by a neural network layer) instead of just raw pixel values.

Over- and Underfitting

When training a model, the goal is to **learn patterns** from the training data that generalize well to **new, unseen data**. But models can go wrong in two main ways:



Underfitting

> The model is **too simple** to capture the underlying structure in the data.

- It performs poorly on both **training** and **validation** data.
- Examples:
  - A red line that cuts straight through a wave-like distribution of blue dots, missing most of the curvature.
  - In the validation graph: **Training and validation loss both stay high**, this means: the model hasn't learned enough.

Overfitting

> The model is **too complex**, learning not just patterns but also **noise** in the training data.

- It performs very well on **training** data but poorly on **validation** data.
- Examples:
  - A red line that wiggles around every data point. It is matching training points exactly but failing to generalize.
  - In the validation graph: **Training loss is low, but validation loss increases**, a clear sign of overfitting.

"Good Fit"

> The model finds a **balanced level of complexity**, capturing the pattern but not memorizing the noise.

- It generalizes well to new data.
- Examples:
  - A smooth red curve that flows **between** the wave-shaped dots.
  - In the validation graph: **Training and validation losses are both low and close together**.

How to Handle Overfitting?

- Use **simpler models**
- Add **regularization**
- Use **more data**
- Try **early stopping**
- **Augment data** or introduce dropout layers in neural nets

> ***What happens when a model overfits to the training data?***
>
> When new data arrives (like fresh spam emails), the model might fail to classify them correctly. It has learned the **training data too specifically** and can't generalize to **unseen cases**.

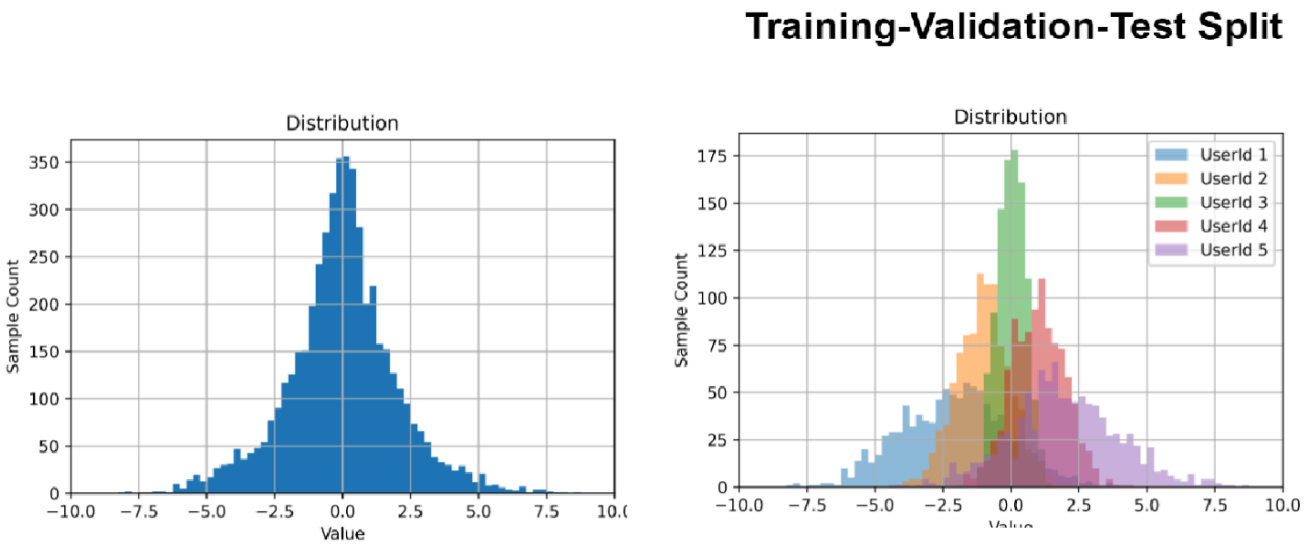
> ***Be careful not to overfit to test sets!***
>
> Optimizing a model too much for a fixed test dataset, it no longer is testing/learning for generalization, it is just fitting another known set.
> In this case, getting a **fresh dataset from the real world** to re-evaluate the model's performance. is a good call.

Training-Validation-Test Split

> **"Train on the known, validate on the unknown, test on the unseen."**
>
> - Training set: A set of examples used for learning, that is to fit the parameters of the classifier.
> - Validation set: A set of examples used to tune the parameters of a classifier, for example to choose the number of hidden units in a neural network.
> - Test set: A set of examples used only to assess the performance of a fully-specified classifier.
>
> Brian Ripley, page 354, Pattern Recognition and Neural Networks, 1996

We will look at data splitting by an example, where we have data from five users, each represented by a color:

- **User 1**
- **User 2**
- **User 3**
- **User 4**
- **User 5**



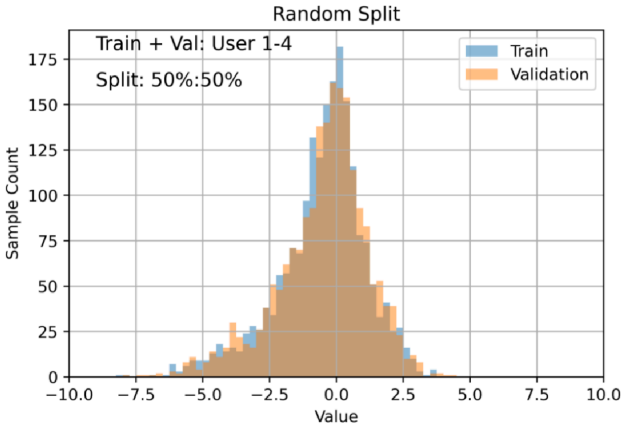
> On the left we see a cumulative set of data samples (*up to ~350 samples*) from different users and on the right we see each user's data seperatly (*so the max y-value is lower*)

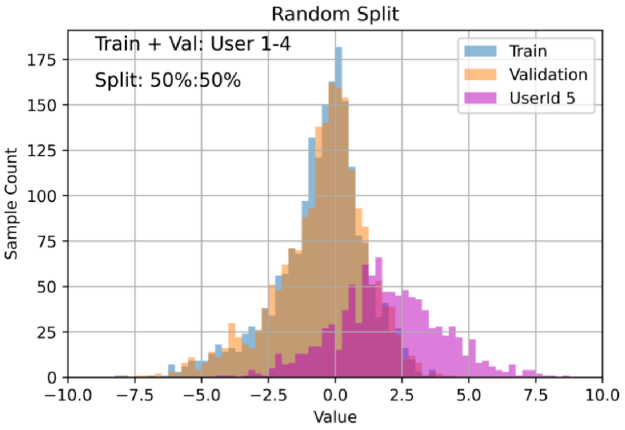
> In general the distribution of values ranges from **-10 to +10** rapidly decreasing sample counts toward the edges!

⚡ The type of distribution dictates how to split the data!

This uneven distribution can make splitting tricky. A naive random split might underrepresent the tails (outliers), which are often the hardest to predict.
Here only 🔵 User 1 and 🟣 User 5 are contributing towars values > 3 *or* < -3

1. Splitting → Random Split Within Participants





Let's assume we apply a **random split across the entire dataset**:

- 50% of the data (across users 1-4) → Training
- 50% of the data (across users 1-4) → Validation
- rest of the data (user 5) → Test

But what happens when we then add User 5 (🟣) as test data?

❓ *Why is a random split across all users problematic in user-specific datasets?*

Because the model might learn to fit data quirks from specific users (1–4), which don't generalize to new, unseen users. This leads to poor real-world performance; especially if User 5 behaves differently or falls outside the training distribution.

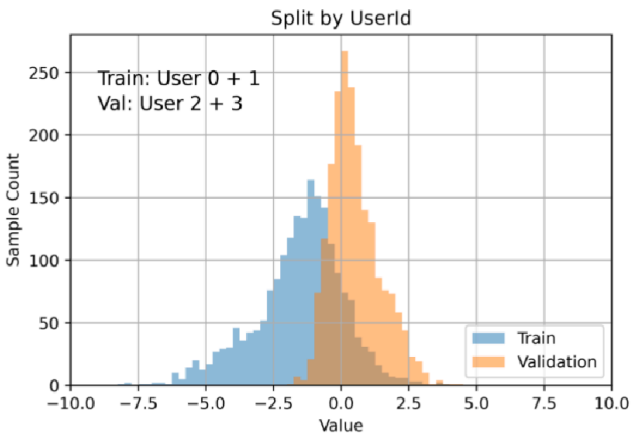
The Distribution Mismatch Problem

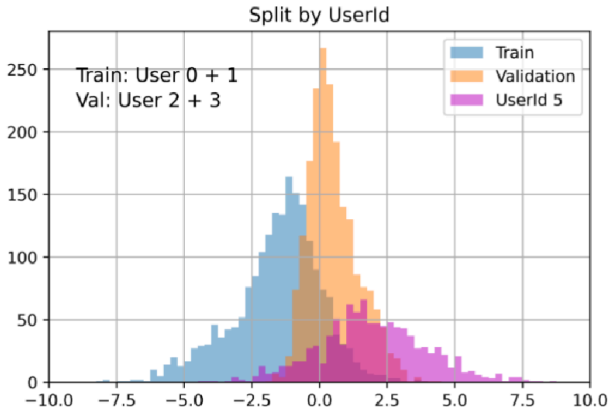
If User 5 has data points with a different distribution (e.g. values in the range -8 to -10, which are rare in the training data), then a model trained on Users 1–4 might not handle them well.

This is a major reason to **not randomly split across users**, but instead split data **participant-wise**.

---

2. Splitting → Participant-wise Split





Now we split the data by user-id also called a **participant-wise split**, that assigns entire users to training, validation, or test sets. For example:

- Users 1+2 → Training
- Users 3+4 → Validation
- User 5 → Test (simulating a real-world unseen user)

This avoids **data leakage** (e.g. having data from the same user in both training and validation).

> 🔍 **Data leakage** occurs when **information from outside the training dataset** is inadvertently used to train the model. Especially information that would **not be available at inference/test time**.

What's the downside of pure participant-wise splitting?

It can lead to **small training sets**, especially in low-data scenarios. Also, the model might underperform simply because it didn't see enough variation.

---

3. Splitting → Participant-wise random Split *(Bonus)*

Instead of splitting users entirely, we could also **split each user's data into train/val/test**. For example:

- 70% of each user's data → Training
- 20% → Validation
- 10% → Test

This increases the amount of data used for training and allows the model to learn from the full distribution ← **within each user**.

Tip

This setup can be helpful for tasks like personalization, where the model needs to generalize across **instances**, not necessarily across **identities**.

When is random split within users preferable?

When we're not concerned with unseen users, but instead want to improve prediction quality **within the same users**, for example in personalized systems.

Summary Table: Splitting Strategies

Strategy	Description	Use Case
Random Split (global)	Randomly splits data across all users	Risk of leakage, not robust to unseen users
Participant-wise Split	Entire users are assigned to train/val/test sets	Testing generalization to **new users**
Random Split within Participants	Each user's data is split individually into train/val/test	Better within-user generalization
Leave-One-Participant-Out	Each user is left out once for testing; training on the others	Robust evaluation across users
Leave-One-Group-Out (e.g. task-based)	Similar to participant-out but for other groupings (e.g. tasks)	Generalization across non-user groups

More training data helps with optimization and generalization.

Backpropagation benefits from more diverse examples: the model can learn a **richer representation** of the task and avoid overfitting to noise.
Especially in participant-wise setups, the **training participants must cover the full range of expected behavior** (e.g. extreme values on both sides of the histogram). Otherwise, the model will struggle with edge cases.

Cross-Validation Variants

- **K-Fold Cross-Validation**: Split data into *k* parts; each fold used once as validation.
- **Leave-k-Out**: Hold out *k* data points each time for testing.
- **Leave-One-Group-Out**: Use one group (e.g. a session, a device) as test set.
- **Leave-One-Participant-Out (LOPO)**: Rotate test users; each user is tested once, trained on the others.

What does LOPO Cross-Validation tell us?

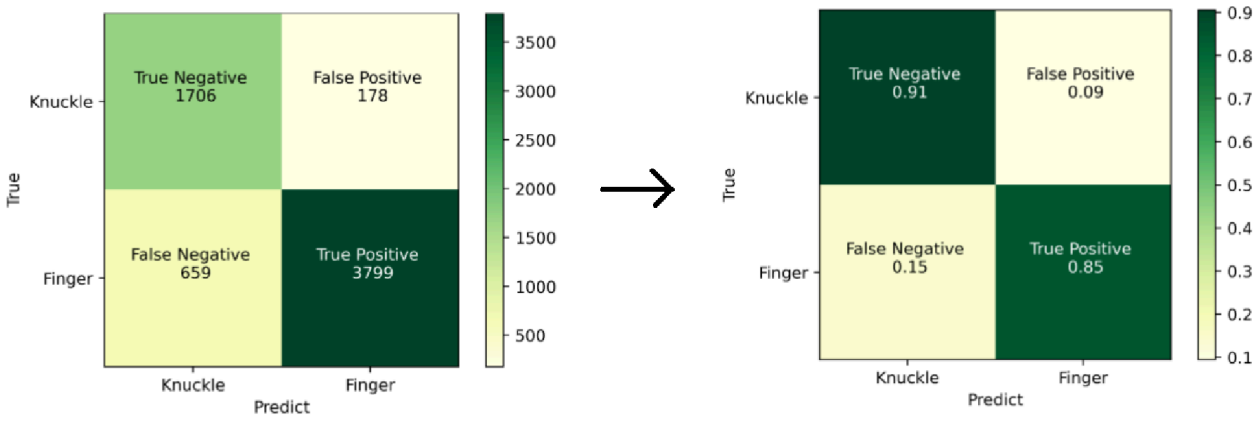
It gives **multiple accuracy scores**, one per participant. If the **standard deviation is low**, it means the model generalizes well across individuals.

Evaluation Metrics & Confusion Matrix

> **"Not all loss functions are good evaluation metrics, and not all evaluation metrics make good loss functions."**

When evaluating a model, especially classifiers, it is often usefull to go **beyond loss functions** like MSE or Cross Entropy, and instead turn to **confusion matrix-based metrics** to understand real-world performance.

Example – Binary Classification (Finger vs. Knuckle)



> This matrix summarizes the model's **actual predictions vs. ground truth**.

> ***Why is it called a "confusion" matrix?***
>
> It shows **where the model is confused**: the off-diagonal elements (FP and FN) reveal where the model makes mistakes.

Raw Counts → Percentages

- We might start with counts like:
  - True positive **(TP)= 3799**
  - **False positive (FP) = 178**
  - False negative **(FN) = 659**
  - True negative **(TN) = 1706**

To turn these into relative values:

$$Presision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 = 2 * \frac{Presision * Recall}{Presision + Recall}$$

$$Accuracy = \frac{TP + TN}{TN + FP + TN + TP}$$

- **Precision** = TP / (TP + FP) → how many predicted Fingers were actually Fingers?
- **Recall** = TP / (TP + FN) → how many actual Fingers were correctly predicted?
- **Accuracy** = (TP + TN) / total samples → how many total predictions were correct?
- **F1 Score** = 2 × (Precision × Recall) / (Precision + Recall) → harmonic mean of precision and recall

What do these metrics really measure?

- **Precision**: Trustworthiness of the prediction
- **Recall**: Coverage of the truth
- **F1 Score**: Balance of both, especially useful in imbalanced datasets

Why prefer F1 over Accuracy in some cases?

Accuracy can be misleading when **class imbalance** is present! That happens, when one class occurs much more frequently than the others.

Example: Detecting fingers vs. knuckle touches, but in the dataset:

- 950 samples are **knuckles**
- 50 samples are **fingers**

If the model always predicts “knuckle” (even when the input is a finger!), it still achieves:

- **Accuracy = 950 / 1000 = 95%**
  This sounds good; but the model is **completely useless** at detecting fingers.

This is why **F1 score** becomes important:

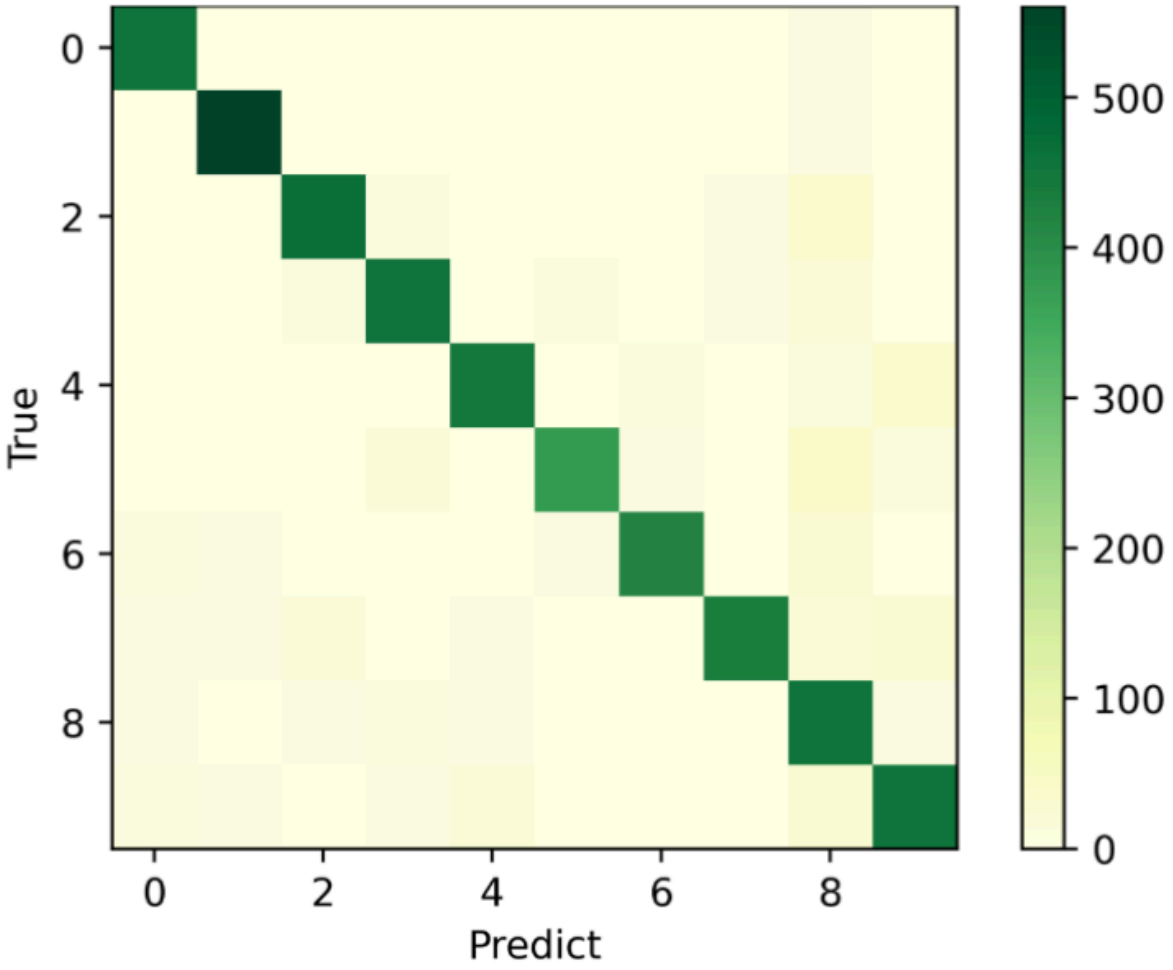
- F1 combines **precision** (how many predicted fingers were actually fingers?) and **recall** (how many actual fingers were correctly detected?)
- In the example above, **precision = 0** and **recall = 0** → so F1 = 0
  → this reflects the model's failure properly
  F1 gives a balanced view of performance on the minority class

⚡ Be careful with accuracy!

Accuracy can be misleading and hide its failure, as it may look high just because the model predicts the **dominant / overrepresented class** (*knuckles*) well, even if it performs poorly on the **rare but also important cases** (*fingers*).

Multi-Class Confusion Matrix

For tasks with >2 classes, the matrix just grows:



In multi-class classification, we aim for **high values on the diagonal**, where predicted label = true label. Off-diagonal elements indicate confusion between classes.

Confusion Matrix vs. Loss Functions

> ***Why not use F1 or accuracy directly as loss functions?***
>
> Because these metrics are **non-differentiable**, and can't be optimized via gradient descent.

Loss functions like:

- **Mean Squared Error (MSE)**
- **Mean Squared Logarithmic Error (MSLE)**
- **Categorical Cross Entropy**

… are **differentiable**, making them suitable for training. But they don't always reflect **real-world performance**, which is why we should usually **evaluate** using precision, recall, F1, etc.

> ***Train with loss, evaluate with metrics.***
>
> Remember: A **Loss function optimizes**, but **metrics explain**.

Hyperparameter Tuning

Hyperparameter tuning is the process of optimizing **model configuration choices** that are not learned from data during training but **must be set before training starts**. These hyperparameters control:

- The **structure** of the model (e.g. number of layers or neurons),
- The **learning process** (e.g. learning rate, optimizer),
- **Regularization techniques** (e.g. dropout),
- And **training behavior** (e.g. batch size, early stopping).

Element	Tunable?	Role in Hyperparameter Tuning
Model structure	yes	Number of layers, hidden units
Loss function	sometimes	Usually fixed, but some variants (e.g. weighted loss) can be tuned.
Optimizer	yes	Choosing between SGD, Adam, RMSProp, etc. affects convergence.
Learning rate	yes	One of the most critical hyperparameters it controls step size in gradient descent.
Dropout rate	yes	Helps prevent overfitting and is tuned to balance regularization.
Batch normalization	kind of	It's usually a design choice, not tuned frequently, but can affect tuning dynamics.

Unlike model parameters (like weights), **hyperparameters are chosen *externally***, often via search and experimentation.

Hyperparameter Search Strategies

Hyperparameter search **requires input data** (e.g., images and labels) to work. Because each combination of hyperparameters needs to be **tested by actually training and evaluating the model**.

Grid Search

Exhaustive search over a **manually defined** set of hyperparameter values.

```
# The function that builds your CNN
def create_model(optimizer='adam', filters=32):
    model = Sequential()
    model.add(Conv2D(filters=filters,
    ...
    ...
    )

# Wraping the model in KerasClassifier
cnn_model = KerasClassifier(model=create_model, epochs=3, batch_size=32, verbose=0)

# The list of hyperparameters we want to test and the values we want to try for each
param_grid = {
  'learning_rate': [0.001, 0.01, 0.1],
  'batch_size': [16, 32, 64]
}

# -------
# Ou **GridSearch** using the given parameters and model
# cv stands for cross-validation. The data will be split into 5 folds: 4 folds used for
training, 1 for validation. This happens 5 times, each time with a different validation fold.
It helps ensure the results are reliable and not just lucky guesses.
sklearn.model_selection.GridSearchCV(estimator=cnn_model, param_grid=params, cv=5)
# -------

# Fit the grid search on the training data
grid_search.fit(X_train, y_train)
```

- Good for **small models** with limited parameter space.
- **Inefficient** for deep learning: exponential growth in combinations.
- Can use **partial grid search** to improve runtime.

> **Tip**
>
> Grid search works best when having **prior knowledge** about useful value ranges.

Random Search

Tests **random combinations** of hyperparameters.

- Often **more efficient** than grid search for high-dimensional spaces.
- Doesn't guarantee best result, but good for early **hyperspace exploration**.
- Works better when only a **few hyperparameters matter a lot**.

> ***Why use random search over grid search?***
>
> Because evaluating every combination in grid search is often wasteful, and random search can find good regions **faster**.

Trial and Error

Manual tuning based on experience and observation.

- Hyperparameters are adapted based on **past failures**.
- Seed values may be **informed or random**.
- Very **common in practice** when deep expertise is available.

> **Info**
>
> Often used in **deep learning** workflows when automated search is too expensive.

Network Growing & Pruning

Growing	Pruning
Start with a small model and **add neurons or layers**. - Less time and resource-intensive at first. - Good for **gradually increasing** capacity.	Start big, then **remove unnecessary neurons**. - Time-intensive but can yield **compact models**. - Suited for **overparameterized** architectures.

> **Tip**
>
> Growing is often better for early-stage tuning. Pruning is useful after a **final model** is trained.

Early Stopping

Stops training when the model **no longer improves** on validation.

```
tensorflow.keras.callbacks.EarlyStopping(patience=5, monitor='val_loss')
```

- Prevents **overfitting** and **wasted computation**.
- Not a tuning method itself, but helps in **hyperparameter evaluation**.

Pre-Trained Models

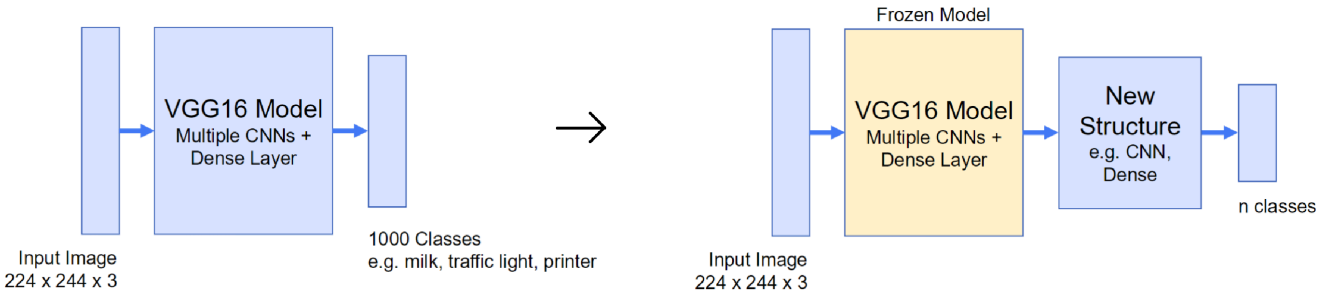
> **Pre Trained Models**
>
> Larger general models help to get started training and reduce training time.

- Good stating point
- Allready have domain knowledge
- Reduces training time
- Reduces computational complexity
- Allows to train with less data – “fine tuning”

VGG16

VGG16 is a deep convolutional neural network from Oxford (16 layers), trained on ImageNet (1.2M images like milk, traffic light, printer,... and can predict 1000 classes).

- Input size: **224x224x3**
- Output: **1000 classes** (e.g. milk, printer, traffic light)



Using VGG16 as a Pre-Trained Model

▪ Reducing the output classes:

Original model (e.g. VGG16) predicts 1000 classes

→ We can adapt it to predict only **e.g. 5 target classes**

▪ Used as **feature extractor**:

Keeping the early layers (CNNs) and remove the classification head

▪ Can be used to then **train a new head** (Dense/CNN layers(...)) to fit **new dataset**

> 🔍 **Freezing a model** = Locking its weights so they don't change during training.
> *Here* we can freeze the VGG16 layers (pre-trained knowledge) and only train **our new custom layers** on top.

Example Code using VGG16 as the Pre Trained Model

```
model = Sequential()

# Input layer
model.add(Input((224, 224, 3), name="Input"))

# Load pre-trained VGG16 without classification head model.add(vgg)  # Add VGG16 as feature
extractor
vgg = tf.keras.applications.VGG16(weights='imagenet', include_top=False)

# Flatten output of VGG
model.add(Flatten())

# New fully connected layer
model.add(Dense(1024, activation='relu'))

# Dropout for regularization
model.add(Dropout(0.5))

# New classification layer for 5 classes
model.add(Dense(5, activation='softmax'))
```

Tip

This is called **Transfer Learning**: using knowledge from one model (ImageNet) and adapting it to a new task (e.g. 5 classes).

Understanding a complete Model (VGG16)

To better understand how VGG16 works this is a good read:
[VGGNet-16 Architecture: A Complete Guide]